

Fyn AI — Tool Catalogue

Date: 2026-04-30 **Branch:** `feature/fyn-persona-split` **Companion to:** `fyn-ai-and-savetax-architecture-map.md`

Every tool the LLM can call across both Fyn personas. Sources of truth:

- `app/Services/AI/AiToolDefinitions.php` — Anthropic schema (1,600 lines)
- `app/Services/AI/XaiToolDefinitions.php` — xAI / OpenAI schema parity (kept tool-name-identical to the Anthropic catalogue per INV-2.7.1)
- `app/Services/AI/AdviceFyn.php::WRITE_TOOLS` — blacklist Advice Fyn never sees
- `app/Services/Onboarding/OnboardingPromptBuilder::toolsForFocus()` — focus-keyed onboarding whitelist
- `app/Services/Onboarding/OnboardingChatDirector::captureToolSet()` — wider whitelist used during inline-capture (handoff path)

There are **47 distinct tools** across 12 functional groups. Counting only what the LLM ever sees in production. Internal-only stubs (`delegate_to_capture`, `capture_complete`) are listed separately.

How the two Fyns see this catalogue

Persona	Source	Tools available
Advice Fyn	<code>AdviceFyn::buildToolList()</code> → full catalogue + handoff tools, then <code>array_diff(self::WRITE_TOOLS)</code>	All read/analysis/billing/plan/tax/handoff tools (~17 tools + <code>delegate_to_capture</code>). Every <code>create_*</code> , <code>update_*</code> , <code>delete_*</code> , <code>capture_*</code> , <code>set_expenditure</code> , <code>navigate_to_page</code> is stripped.
Onboarding Fyn — <code>delegated/asset_capture</code> turn	<code>OnboardingPromptBuilder::toolsForFocus(\$focus)</code>	1–8 focus-specific <code>create_*</code> / <code>capture_*</code> tools + <code>update_profile</code> + <code>update_record</code> . NO read tools, NO navigation.
Onboarding Fyn — <code>grouped_extract</code> turn	<code>toolsListOverride</code> (replaces full catalogue)	One narrow extraction tool per state: <code>capture_personal_details</code> , <code>capture_spouse_details</code> , <code>capture_dependants</code> , or <code>capture_work_details</code> . Nothing else.
Onboarding Fyn — inline-capture (handoff target from Advice Fyn)	<code>OnboardingChatDirector::captureToolSet(CaptureContext)</code>	Wider whitelist: every <code>create_*</code> plus <code>update_record</code> , <code>update_profile</code> , <code>set_expenditure</code> , <code>delete_record</code> , <code>create_what_if_scenario</code> , plus the four SaveTax <code>capture_*</code> tools.

The advice path can ALSO short-circuit to inline-capture deterministically (no LLM) via `WriteIntentClassifier` — in which case the LLM never picks tools at all; the `RecordDuplicateChecker` and inline-capture handler do the work.

1. Read tools (Advice Fyn only)

These are the tools `AdviceFyn` exposes to the LLM. None are write tools, none are in `WRITE_TOOLS`.

`navigate_to_page` — *stripped from advice in S0.5.t*

STATUS: blacklisted in `WRITE_TOOLS`. Was in the navigation group but no longer reachable from Advice Fyn. The S0.5.t hardening removed it because BS-14 caught the LLM using it as an escape hatch for write intents (sending the user to a page so they fill the form themselves)

and then fabricating "I've added X" success text. Advice has no navigation; users navigate via the menu. Onboarding never has it either.

list_records

Lists existing records of a given type with IDs, key details, balances, interest rates, and values. Used BEFORE `update_record` to find the correct `entity_id`, AND for factual questions about the user's accounts (balances, interest rates, providers, policy details). Returns raw data; `get_module_analysis` is the heavier alternative.

Param: `entity_type` (enum) — `savings_account`, `investment_account`, `dc_pension`, `db_pension`, `property`, `mortgage`, `life_insurance`, `critical_illness`, `income_protection`, `trust`, `business_interest`, `chattel`, `estate_liability`, `estate_gift`, `family_member`.

list_goals

Lists all of the user's financial goals with current progress, status, and IDs. Lightweight alternative to `get_module_analysis(goals)` when the goal list is enough.

No params.

list_life_events

Lists all life events with dates, amounts, and IDs. Lightweight alternative to `get_module_analysis(goals)` when only the event list is needed.

No params.

get_module_analysis

Returns full personalised analysis for a single module. Heavy call — runs the relevant `{Module}Agent::analyze()`. Used when the LLM needs computed metrics (coverage gap, emergency fund adequacy, portfolio drift, IHT liability, etc.), not just a record list.

Param: `module` (enum) — `protection`, `savings`, `investment`, `retirement`, `estate`, `goals`, `holistic`. The `holistic` value calls `CoordinatingAgent::orchestrateAnalysis` and runs all 9 module agents — only `HOLISTIC_HEALTH` queries hit this branch under the F-10/F-15 sizing rules.

get_recommendations

Returns the user's personalised financial recommendations ranked by priority across all modules. Decision-tree triggered.

No params.

search_conversation_index

Searches the user's prior conversations for context on a topic or entity. Returns up to 10 prior conversations matching the supplied keywords/entity types, ordered by recency. Used ONLY when the `<known_facts>` block is silent on the field needed AND the user references a previous session ("as we talked about last time"). NOT a substitute for `list_records` or `get_module_analysis` — those return current authoritative data; this returns historical conversational context. Backed by the `ai_conversation_index` table populated by `ConversationSummariserJob`.

Params:

- `topic_keywords` (array of strings) — module-level topic tags: `protection`, `savings`, `investment`, `retirement`, `estate_planning`, `goals_life_events`, `tax_optimisation`, `family`, `property`, `mortgage`, `billing`, `general`.
- `entity_types` (array of strings) — entity types from the `entities_mentioned` field: `life_insurance_policy`, `dc_pension`, `db_pension`, `isa`, `gia`, `savings_account`, `property`, `mortgage`, `credit_card`, `family_member`, `goal`, `life_event`, `will`, `trust`, `business_interest`, `chattel`.

get_tax_information

Gets current UK tax-year information for a specific topic. ALWAYS used when the user asks about tax thresholds, allowances, rates, or any financial product tax treatment. The LLM is instructed never to state tax values from memory — always retrieve them. Backed by `TaxConfigService`.

Param: `topic` (enum) — `income_tax`, `national_insurance`, `capital_gains`, `dividend_tax`, `inheritance_tax`, `gifting_exemptions`, `stamp_duty`, `isa_allowances`, `pension_allowances`, `state_pension`, `benefits`, `savings_config`, `assumptions`, `investment_bonds`, `venture_capital`, `protection_config`, `retirement_config`, `domicile`, `income_definitions`. Use `income_definitions` to get the user's adjusted net income, threshold income, and tapered pension allowances.

generate_financial_plan

Generates a comprehensive holistic financial plan: executive summary, top recommendations, overall score, and action plan. Analyses all modules. Used when the user asks for a financial plan, overview of position, or what to prioritise.

No params.

get_subscription_status

Gets the user's current subscription status — plan, billing cycle, current period end, trial end, next charge, and whether they have cancelled. Used when the user asks about their subscription, billing, when they will be charged next, whether their trial has ended, or whether their subscription is still active. Auto-emits a Subscription Management CTA card from the tool result (frontend consumes the **action:navigate** field).

No params.

list_invoices

Lists the user's invoices in reverse chronological order. Each row includes invoice number, issued date, amount in pounds, currency, status, plan name, billing cycle, and a PDF download URL. Used for billing history, past invoices, or download-receipt requests.

No params.

get_current_plan

Gets the details of the user's current subscription plan — name, tier slug, billing cycle, price in pounds, and feature list. Used when the user asks what plan they are on, what features they have, or what they are paying.

No params.

2. Direct-write tools — life records (Onboarding Fyn only; stripped from Advice Fyn)

Every tool below is in **AdviceFyn::WRITE_TOOLS** and therefore filtered out of the advice tool list. Onboarding sees them via **toolsForFocus** (asset_capture turns) or **captureToolSet** (inline-capture from a delegated handoff). Each handler runs in its own transaction, fires observers, writes an **ai_audit_events** row via **appendAuditCompletion**, and (post-F-9) invalidates **AdvicePromptCacheInvalidator::forUser** on success.

The multi-entity rule in the onboarding prompt instructs the LLM to emit one **tool_use** per record in the SAME assistant turn ("Halifax ISA £10k AND Nationwide saver £5k" → **create_savings_account** × 2, not two separate turns).

create_savings_account

Creates a savings account. Cash ISAs, easy-access savers, fixed-rate accounts, current accounts. Multi-call enabled.

create_investment_account

Creates an investment account. Covers ISAs (Stocks & Shares ISA), GIAs, bonds, VCT, EIS, private company shares, crowdfunding, employee share schemes (SAYE, CSOP, EMI, share options, RSUs), and other investments. Accepts an optional **holdings** parameter — when present, the holdings are attached to the new account in the same transaction (NOT a separate **create_holding** call). Multi-call enabled.

create_holding

Adds a holding to an EXISTING investment account that was created without holdings. ONLY when the account already exists. If creating a NEW account with holdings, use **create_investment_account** with the **holdings** parameter instead. Multi-call enabled.

create_pension

Creates a pension. Handles both Defined Contribution (workplace, SIPP, personal pension) and Defined Benefit (final salary, career average) types. Multi-call enabled.

create_property

Creates a property. If the user mentions a mortgage at the same time, **outstanding_mortgage_amount** parameter triggers automatic mortgage creation in the same transaction. Multi-call enabled.

Anti-pattern enforced in the prompt: do NOT call `navigate_to_page` or `get_module_analysis` in the same turn as `create_property` — those interrupt the form fill.

`create_mortgage`

Creates a standalone mortgage linked to an existing property. Used when the user mentions a mortgage separately from a property, or wants to add a mortgage to an existing property. Multi-call enabled.

`create_protection_policy`

Creates a protection insurance policy. Handles life insurance, critical illness cover, and income protection. Multi-call enabled (e.g. life insurance AND critical illness in the same message).

`create_asset`

Creates a generic asset — collectibles, artwork, or other valuables not covered by other tools. Multi-call enabled.

`create_liability`

Creates a liability. Covers credit cards, personal loans, student loans, car finance, or any outstanding balance owed. Multi-call enabled.

`create_chattel`

Records a personal valuable item (jewellery, art, collectibles, vehicles). Multi-call enabled.

`create_business_interest`

Records a business interest or ownership — limited companies, partnerships, self-employment assets. Multi-call enabled.

`create_family_member`

Adds a family member (spouse, child, dependant). Multi-call enabled — for two children, the LLM should call `create_family_member` TWICE in the same turn.

3. Direct-write tools — estate planning

`create_estate_gift`

Records a gift for Inheritance Tax planning. Affects the user's IHT position under the 7-year rule. Multi-call enabled.

`create_trust`

Records a trust for estate planning. Multi-call enabled.

`create_will`

Records the user's existing will (executor, beneficiaries, guardians, specific gifts). Used when the user tells Fyn they ALREADY HAVE a will. The Will Builder UI remains the tool for drafting a new will from scratch.

`update_will`

Updates an existing will record (new executor, new beneficiary, updated specific gifts).

`create_power_of_attorney`

Records a Lasting Power of Attorney (LPA) the user already has. UK has two types: Property & Financial Affairs (`property_financial`) and Health & Welfare (`health_welfare`). Captures `primary_attorney_name`, optional `replacement_attorney_name`, mandatory `status` (`draft` | `registered`), and optional `opg_reference`.

Status extraction rules (from the tool's prompt):

- "registered" / "in force" / "active with OPG" / "already registered with the Office of the Public Guardian" → `status='registered'`
- "draft" / "signed but not registered" / "in the pipeline" / "pending registration" → `status='draft'`
- No signal → default `status='draft'`

Multi-call enabled — if the user has BOTH a `property_financial` AND a `health_welfare` LPA, call twice.

update_power_of_attorney

Updates an existing LPA record (status change from draft to registered, OPG reference added, replacement attorney added).

4. Direct-write tools — goals & scenarios

create_goal

Creates a financial goal — house deposit, holiday fund, retirement target, emergency fund, etc. Captures name, target amount, target date, linked accounts. Multi-call enabled.

create_life_event

Records a future life event that may impact the financial plan (career change, house move, child, retirement). Multi-call enabled.

create_what_if_scenario

Creates a persistent what-if scenario showing how a change would affect the plan. The scenario is saved as a **WhatIfScenario** row. Used for "what if I retire at 55", "what if I sell my main residence", etc. **Reachable only via the inline-capture path** from advice — Advice Fyn cannot call this directly because it persists DB state (S0.5.r tightened this; pre-S0.5.r it was an analytics carve-out).

5. Modification & deletion

update_record

Updates an existing record. Used for "change my Cash ISA balance to £15k", "my pension provider is now Vanguard", retracting earlier captures, etc. The schema enforces a strict per-**entity_type** field allowlist via **UpdateRecordAllowlist** — invented field names are rejected at handler time. **The LLM must call `list_records` first** to find the correct **entity_id**. Multi-call enabled.

Per INV-2.7.3: Anthropic uses a true **oneOf** discriminated union schema; xAI uses a flatter union schema (xAI strict-mode limitation). The runtime allowlist enforces the same field policy on both.

delete_record

Deletes an existing record with **two-phase confirmation**:

1. First call → returns a **confirmation_token** and **preview_message**. The LLM is instructed NOT to delete on this turn — show the preview message and ask the user to confirm.
2. Second call with the EXACT token echoed back → deletion proceeds.

Tokens are bound to (**user**, **entity_type**, **entity_id**, **today's date**) via day-salted HMAC + **hash_equals**. They cannot be replayed across days, and a stolen token from yesterday's session cannot delete today.

6. Profile & expenditure

update_profile

Updates personal profile information — age, income, expenditure, marital status, address, domicile. Used when the user provides personal info during normal conversation. Available in onboarding (in every focus tool list as the retraction tool) AND inline-capture.

set_expenditure

Sets the user's monthly expenditure by category. Captures all categories the user mentions in a SINGLE call (this tool does NOT support multi-call — calling it twice in a turn overwrites). Categories: housing, transport, groceries, utilities, etc. Omit categories the user didn't mention.

7. Onboarding base-flow extraction tools

Used during `grouped_extract` turns of the structured base flow. Replace the entire tool list (via `toolsListOverride`) — the LLM sees ONLY the one extraction tool for that state. Used by `BASE_PERSONAL`, `BASE_SPOUSE`, `BASE_DEPENDANTS`, and `BASE_WORK` states.

`capture_personal_details`

Captures the user's date of birth and/or marital status from a free-text reply. **Strict prompt rule:** only include a field when the user has EXPLICITLY stated it. Do not guess, infer, or default. Omit fields entirely rather than inventing values — the onboarding flow re-asks for anything missing.

`capture_spouse_details`

Captures the user's spouse or civil partner details and creates a linked spouse user account.

`capture_dependants`

Captures a list of the user's dependants (children or other dependants) — array of all dependants mentioned in the message.

`capture_work_details`

Captures the user's employer, position, and annual income. Only used when the user is employed, self-employed, or part-time.

8. SaveTax campaign-specific capture tools

Used during the `CAMPAIGN_*` states of the SaveTax onboarding flow. Whitelisted by both `toolsForFocus('savetax')` and `captureToolSet`. The state machine itself gates which campaign state can call which tool.

`capture_salary_sacrifice`

Sets the `salary_sacrifice` flag on a specific DC pension owned by the user. Used during the `CAMPAIGN_OCCUPATIONAL_SCHEME` state. Takes the `dc_pension_id` and a boolean.

`capture_spouse_work_status`

Sets whether the user's spouse currently works. Updates `users.household_calculation_mode` (`dual_earner` | `single_earner_couple`) and `users.marriage_allowance_eligible` accordingly. The state machine routes the next state based on the result — `dual_earner` → `CAMPAIGN_SPOUSE_HOUSEHOLD`; `single_earner_couple` → `CAMPAIGN_SPOUSE_NON_WORKING_ASSETS`.

`capture_spouse_household_data`

Captures working-spouse data for `dual_earner` households (the `spouse_works=yes` path). Writes to `tax_strategy_household_inputs` — spouse income, ISA balance, PSA band, unrealised gains, dividends, pension input, etc.

`capture_spouse_non_working_assets`

Captures standalone assets owned by a non-working spouse (the `single_earner_couple` path). Used to compute capacity for asset-shifting strategies — Personal Allowance, Starting Rate for Savings, Personal Savings Allowance, ISA, CGT, and Dividend allowance.

9. Internal handoff tools (never user-visible)

Defined in `app/Services/AI/HandoffContract.php` and emitted via `AiToolDefinitions::handoffTools($provider)`. These are the contract that lets the two-Fyn split look like a single chat surface to the user.

`delegate_to_capture` (Advice Fyn → Onboarding Fyn)

Internal. Emitted by Advice Fyn when the LLM determines the user wants to add/save/record/update/delete a persistent record (the only path Advice Fyn has to write — it has zero direct-write tools). `CoordinatingAgent::executeTool` translates the call into a synthetic handoff SSE event with `handoff_type='delegate_to_capture'`. `AdviceFyn::wrapStream` intercepts the event, validates via `HandoffPayloadValidator::validateDelegateToCapture` (post-F-1), builds a `CaptureContext` from the payload, and `yield` from `OnboardingChatDirector::handleInlineCapture`. The `handoff` event itself is **stripped from the SSE stream** (INV-2.4.1) — never reaches the frontend.

Required payload:

- **reason** (string) — one-sentence why, e.g. "User wants to add a Cash ISA at Nationwide." Soft-required (post-F-1): missing **reason** is recovered via `CaptureContext::fromArray` synthesis from **entity_types**. Logged at notice level.
- **entity_types** (array of strings) — record types, e.g. `["savings_account"], ["protection_policy"]`. Hard-required — missing or non-array → **handoff_error** SSE event + **done** (post-F-1).

Optional:

- **fields_needed** (array of strings) — field names the user provided.
- **pending_advice_question** (string) — set when capture was triggered because advice needed data; advice resumes with this question after capture completes.
- **originating_focus** (string) — onboarding journey focus that triggered the capture.

capture_complete (Onboarding Fyn → Advice Fyn)

Internal. Emitted by Onboarding Fyn (specifically `handleInlineCapture`) when capture is finished. Frontend consumes the event as a record-card bubble (`commit('ADD_MESSAGE', {role: 'capture_complete', ...})` in `aiChat.js`). The bubble shows the saved records inline with a "Saved to your records" / "Saved N records" heading. NOT shown to the user as Fyn switching modes — visually it's just a normal Fyn message with a record card.

Required payload:

- **summary** (string) — one-line heading.
- **records_created** (array) — list of `{type, id, name}` for each persisted record.

10. Per-Fyn tool matrix

Tool	Advice Fyn	Onboarding (focus-filtered asset_capture)	Onboarding (grouped_extract)	Onboarding (inline-capture)
<code>navigate_to_page</code>	✗ stripped (S0.5.t)	✗	✗	✗
<code>list_records</code>	✓	✗	✗	✗
<code>list_goals</code>	✓	✗	✗	✗
<code>list_life_events</code>	✓	✗	✗	✗
<code>get_module_analysis</code>	✓	✗	✗	✗
<code>get_recommendations</code>	✓	✗	✗	✗
<code>search_conversation_index</code>	✓	✗	✗	✗
<code>get_tax_information</code>	✓	✗	✗	✗
<code>generate_financial_plan</code>	✓	✗	✗	✗
<code>get_subscription_status</code>	✓	✗	✗	✗
<code>list_invoices</code>	✓	✗	✗	✗
<code>get_current_plan</code>	✓	✗	✗	✗
<code>create_savings_account</code>	✗	✓ (savings, budgeting, savetax)	✗	✓
<code>create_investment_account</code>	✗	✓ (investment, savetax)	✗	✓
<code>create_holding</code>	✗	✓ (investment, savetax)	✗	✓
<code>create_pension</code>	✗	✓ (retirement, savetax)	✗	✓
<code>create_property</code>	✗	✓ (estate)	✗	✓
<code>create_mortgage</code>	✗	✗	✗	✓
<code>create_protection_policy</code>	✗	✓ (protection)	✗	✓
<code>create_asset</code>	✗	✓ (estate)	✗	✓
<code>create_liability</code>	✗	✓ (estate)	✗	✓

Tool	Advice Fyn	Onboarding (focus-filtered asset_capture)	Onboarding (grouped_extract)	Onboarding (inline-capture)
create_estate_gift	✗	✓ (estate)	✗	✓
create_chattel	✗	✓ (estate)	✗	✓
create_business_interest	✗	✓ (business)	✗	✓
create_family_member	✗	✗	✗	✓
create_will	✗	✗	✗	✓
update_will	✗	✗	✗	✓
create_power_of_attorney	✗	✗	✗	✓
update_power_of_attorney	✗	✗	✗	✓
create_trust	✗	✗	✗	✓
create_goal	✗	✓ (goals)	✗	✓
create_life_event	✗	✗	✗	✓
create_what_if_scenario	✗	✗	✗	✓
update_record	✗	✓ (every focus — retraction tool)	✗	✓
delete_record	✗	✗	✗	✓
update_profile	✗	✓ (every focus — retraction tool)	✗	✓
set_expenditure	✗	✗	✗	✓
capture_personal_details	✗	✗	✓ (BASE_PERSONAL)	✗
capture_spouse_details	✗	✗	✓ (BASE_SPOUSE)	✗
capture_dependants	✗	✗	✓ (BASE_DEPENDANTS)	✗
capture_work_details	✗	✗	✓ (BASE_WORK)	✗
capture_salary_sacrifice	✗	✓ (savetax)	✗	✓
capture_spouse_work_status	✗	✓ (savetax)	✗	✓
capture_spouse_household_data	✗	✓ (savetax)	✗	✓
capture_spouse_non_working_assets	✗	✓ (savetax)	✗	✓
delegate_to_capture	✓ (always)	✗	✗	✗
capture_complete	✗	✗	✗	✓ (emitted on completion)

Counts:

- Advice Fyn sees: **12 read tools + delegate_to_capture = 13 callable tools.**
- Onboarding focus-filtered asset_capture sees: **2–8 tools per focus** (always includes `update_profile` + `update_record`).
- Onboarding grouped_extract sees: **exactly 1 tool** per state (replaces the full catalogue).
- Onboarding inline-capture sees: **24 tools** — every direct-write the user can trigger mid-session.

toolsForFocus map (asset_capture turns)

From `OnboardingPromptBuilder::toolsForFocus($focus)`. `update_profile` and `update_record` are appended to every focus.

Focus	Tools
savings, budgeting	create_savings_account
investment	create_investment_account, create_holding

Focus	Tools
retirement	create_pension
protection	create_protection_policy
estate	create_asset, create_liability, create_estate_gift, create_property, create_chattel
business	create_business_interest
goals	create_goal
savetax	create_pension, capture_salary_sacrifice, create_savings_account, create_investment_account, create_holding, capture_spouse_work_status, capture_spouse_household_data, capture_spouse_non_working_assets
(default fallback)	create_savings_account

11. Preview-mode behaviour

When `users.is_preview_user = true`, `AiToolDefinitions::getTools(true)` skips `whatIfTools`, `dataCreationTools`, `additionalCreationTools`, `dataModificationTools`, `profileTools`, `expenditureTools`, and `campaignSaveTaxTools`. Preview users see ONLY the `read/analysis/billing/plan/tax` tools regardless of which Fyn is active.

The `AdvicePromptBuilder` Layer 12 also adds a `<preview_mode>` block instructing the LLM never to emit `delegate_to_capture` and to surface a sign-up CTA instead.

The preview filter is **bypassed** by the eval flow ONLY when both conditions hold (post-F-12):

1.

The active Sanctum token has the `bypass-preview-mode` ability (issued by `EvalAuthController::login`, gated to non-production)
2.

The request carries a non-empty `X-Eval-Run-Id` header

Wired at three checkpoints: `HasAiChat::chat` (tool list), `CoordinatingAgent::executeTool` (tool dispatch), and `PreviewWriteInterceptor` middleware (HTTP-level write block).

12. Tool result shape contract

Every tool handler returns an array conforming to `ToolResultContract`. The shapes the SSE stream cares about:

Result shape	Triggers SSE event	Notes
{action: 'navigate', route_path, description}	navigation	Navigates the user. Auto-emitted by <code>get_subscription_status</code> .
{action: 'fill_form', entity_type, route, fields, mode, entity_id?}	fill_form	Pre-S0.5 pattern — opens a form. No tool currently emits this; deprecated.
{action: 'handoff', handoff_type, payload}	handoff	INTERNAL only — never forwarded to the frontend (INV-2.4.1).
{created: true, entity_type, entity_id, name}	entity_created	Direct-write success.
{onboarding_capture: true, field_group, summary, details}	onboarding_field_captured	grouped_extract success.
{onboarding_capture_error: true, field_group, error_type, message}	onboarding_capture_error	grouped_extract domain error (e.g. spouse email already bound).
{error: true, ...}	(no event — model surfaces it)	Tool error. Per FCA rule, the model must surface failures verbatim — never claim "I've added X" on error.

After F-3 (post-audit), tool results are **compressed** via `HasAiChat::compressToolResultForModel` before being re-injected into the LLM message history:

- Errors pass through verbatim (the model needs to surface them).
- Handoff / navigate / fill_form actions pass through.
- Direct-write results trim to {success, entity_type, entity_id, name} (the chaining surface only).
- Read tools recursively trim list arrays >10 items, depth ≥3, strings >200 chars.

The `summariseToolResult` (audit-row metadata) is a separate compression pass that always preserves `entity_id` + `entity_type` (INV-2.5.3).

Tool catalogue current as of 2026-04-30 post-audit (F-1 through F-15 applied). Counts: 47 distinct tools + 2 internal handoff tools = 49 total entries the LLM might ever interact with across both providers.